

SecureFace: A Liveness-Aware Face Recognition Security System with Challenge-Response Verification, Local Deployment Guidance, and Audit Logging

Rachith Bharadwaj T N, Ullas G M, M G Shree Harsha

IIIT Dharwad

24BDS062, 24BDS085, 24BDS037

Abstract

This report presents a detailed repository-level study of *SecureFace*, a face-recognition security system designed for live biometric registration and authentication. The inspected project combines a Flask-based web dashboard, browser webcam capture, randomized challenge-response verification, a Vision Transformer liveness model, Buffalo / InsightFace face embeddings, SQLite-backed audit logging, and unknown-visitor review. The repository also preserves an earlier offline experimental pipeline based on a custom CelebA embedding CNN, revealing the project evolution from research notebooks to an interactive liveness-aware authentication system. In addition to the architecture and model discussion, this report includes a complete deployment section for running the demo on a separate PC/laptop, mirroring the provided README: hardware requirements, software requirements, installation steps, runtime commands, troubleshooting guidance, and dashboard screenshots captured from the locally running application. The inspected snapshot already contains enrolled users, stored profile embeddings, and audit records, demonstrating that the system has progressed beyond a toy prototype into a functioning local demo environment.

Keywords: biometric authentication, face recognition, liveness detection, anti-spoofing, Flask, InsightFace, SQLite, computer vision security

1 Introduction

Face-based access control is attractive because it is contactless, intuitive, and deployable using commodity hardware. However, recognition alone is not enough for secure authentication. A practical system must also resist replay attacks, printed photo attacks, synthetic-face attacks, and deepfake-like presentations. It must record security events, distinguish unknown humans from spoof attempts, and remain simple enough to demonstrate on a normal laptop.

The SecureFace project addresses this combined problem. The current repository implements a local web application that collects live webcam frames, asks the user to perform a randomized liveness challenge, analyzes the captured session, matches it against enrolled identities, and logs the result. The project further includes older training notebooks, threshold sweep utilities, and an offline single-video pipeline, allowing both implementation review and development-history analysis.

This report was prepared by reading the project in detail, running the local dashboard, and capturing screenshots from the actual deployed interface. No architectural claims below are invented; they are based on the repository snapshot and observed runtime behavior.

2 Repository Review and Current State

2.1 Major Components

The repository combines deployable application code, stored model artifacts, and research notebooks. Table 1 summarizes the most important components.

2.2 Observed Local Runtime State

The inspected repository already contains meaningful local state rather than only source code. Table 2 summarizes the observed snapshot.

The profile files indicate that the currently deployed web path stores 512-dimensional embeddings, which is consistent with the Buffalo model rather than the older 256-dimensional CelebA CNN embedding model.

3 System Architecture and Workflow

3.1 Frontend Behavior

The current demo uses Flask templates for the interface and browser-side JavaScript for live webcam capture. The dashboard offers five main views:

Table 1: Main Repository Components

Artifact	Type	Purpose
<code>video_auth_webapp.py</code>	Flask web app	Exposes routes for home, registration, authentication, audit view, unknown-alert view, and CSV exports.
<code>run_dashboard.py</code>	Runner	Starts the dashboard by importing and serving the Flask app.
<code>buffalo_auth_core.py</code>	Core runtime engine	Implements challenge validation, face extraction, liveness inference, Buffalo embedding matching, SQLite storage, and audit operations.
<code>video_auth_pipeline.py</code>	Offline pipeline script	Runs a video authentication experiment using the liveness model and a custom CelebA CNN embedding checkpoint.
<code>templates/</code>	Frontend templates	Contains the Home, Register, Authenticate, Result, Audit, and Unknown Alerts pages.
<code>face_liveness_vit/</code>	Liveness package	Stores the custom ViT checkpoint, YAML config, README, and helper inference script.
<code>buffalo/</code>	ONNX model pack	Local InsightFace Buffalo models for detection, alignment, landmarks, gender-age, and recognition.
<code>checkpoints/</code>	CNN artifacts	Stores the custom CelebA embedding checkpoint family and a validation embedding cache.
<code>app_data/</code>	Local runtime data	Stores the SQLite database, uploaded session data, and registered profile files.
<code>threshold_sweep_eval.py</code> and <code>exact_pair_sweep.py</code>	Evaluation scripts	Evaluate genuine, impostor, and spoof performance under different thresholds.
Notebook files	Research assets	Preserve training, inference, Buffalo comparison, and embedding-consistency experiments.

Table 2: Observed Local Runtime State

Item	Observed State
Enrolled users	3 users: Chinmay, Rachith, Tejas
Profile embedding rows in SQLite	30
Stored .npz profile files	3
Audit events	10
Unknown-person alerts	0
Evaluation manifest	24 clips total
Real / spoof clip split	12 real, 12 spoof
Identities in manifest	8 identities
Committed raw evaluation videos	Not present in local <code>video/real</code> or <code>video/spoof</code> folders
Committed threshold sweep reports	Scripts present, report outputs not present

- Home,
- Register,
- Authenticate,
- Audit,
- Unknown Alerts.

The Register and Authenticate pages require live webcam interaction. Before any frames are accepted, the user must complete a randomized challenge:

- blink,
- turn the face slightly left,
- turn the face slightly right.

Client-side challenge tracking uses MediaPipe facial landmarks [6] to estimate eye behavior and face turning. The interface automatically submits the frame bundle after enough usable frames are captured. The application is intentionally privacy-oriented: failed authentication does not reveal the enrolled identity that scored closest internally.

3.2 Backend Workflow

The backend processing flow in the deployed engine is:

1. issue a random challenge and save a challenge nonce in session,

2. receive webcam frames,
3. validate CSRF token and challenge nonce,
4. verify challenge completion,
5. detect and align face crops,
6. evaluate session liveness,
7. compare the query against all enrolled galleries,
8. write audit records and render the result page.

The deployed web application explicitly rejects direct video uploads and only accepts live webcam frames for the user-facing registration and authentication routes.

3.3 Decision Logic

For each usable face crop, the liveness model produces a spoof probability p_i . Let τ_l be the liveness threshold. A frame is considered live when

$$\text{LiveFrame}_i = \mathbb{K}(p_i < \tau_l). \quad (1)$$

For a session of N valid face crops, the session statistics are

$$\bar{p} = \frac{1}{N} \sum_{i=1}^N p_i, \quad r_{\text{live}} = \frac{1}{N} \sum_{i=1}^N \text{LiveFrame}_i. \quad (2)$$

The session passes liveness if

$$\text{SessionLive} = \mathbb{K}(\bar{p} < \tau_l \wedge r_{\text{live}} \geq \rho_l), \quad (3)$$

where ρ_l is the configured minimum live-frame ratio.

For identity matching, each query embedding $\mathbf{q}_i$ is compared against a gallery of enrolled embeddings $\mathbf{g}_j$:

$$s_i = \max_j (\mathbf{q}_i^\top \mathbf{g}_j). \quad (4)$$

The session-level identity statistics are

$$S = \text{median}(s_i), \quad r_{\text{match}} = \frac{1}{N} \sum_{i=1}^N \mathbb{K}(s_i \geq \tau_m), \quad (5)$$

where τ_m is the similarity threshold. The final decision becomes

$$\text{Authenticated} = \mathbb{K}(\text{SessionLive} \wedge S \geq \tau_m \wedge r_{\text{match}} \geq \rho_m). \quad (6)$$

3.4 Dashboard Images

The screenshots in Fig. 1 were captured from the locally running demo by launching `python run_dashboard.py` and loading the corresponding routes in a browser.

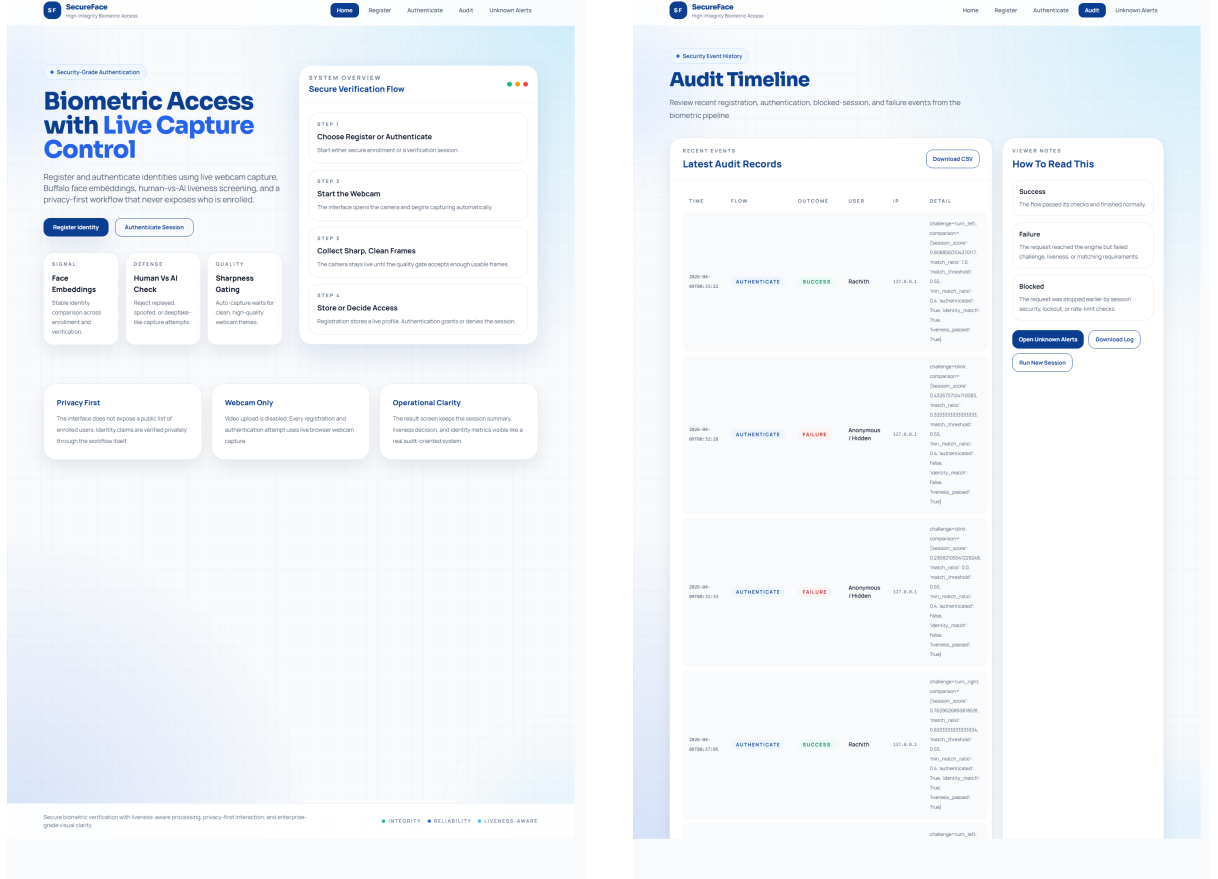


Figure 1: Local screenshots captured from the running SecureFace dashboard.

4 Models and Experimental Evolution

4.1 Liveness Model

The liveness detector is a compact Vision Transformer [1] stored in `face_liveness_vit/model.pt`. The training notebooks and checkpoint-loading utilities are implemented in PyTorch [7]. The associated configuration and model card indicate:

- input size: 224×224 ,
- patch size: 16,
- embedding dimension: 128,
- 4 attention heads,

- 2 transformer layers,
- 2 output classes: live and spoof,
- training dataset: CelebA-Spoof [4],
- optimizer: AdamW,
- 30 training epochs,
- stored optimal threshold: approximately 0.1199.

4.2 Older Custom Embedding Model

The repository also includes a custom face embedding CNN trained on CelebA identities using a ResNet18 backbone [2]. The stored checkpoint metadata indicates:

- embedding size: 256,
- image size: 224,
- number of classes: 10,177,
- best saved checkpoint at epoch 20,
- validation accuracy near 50.34%.

This older embedding model is still used by the offline script `video_auth_pipeline.py`, but not by the currently deployed web dashboard.

4.3 Current Identity Stack

The deployed web demo uses Buffalo / InsightFace [3] rather than the custom CNN for identity matching. This stack belongs to the same modern deep face-recognition family popularized by ArcFace-style identity learning [11]. This is evidenced by:

- the Buffalo ONNX model pack stored in `buffalo/`,
- the runtime code path in `buffalo_auth_core.py`,
- the 512-dimensional profile embeddings saved in `app_data/profiles/`.

This makes the repository especially interesting: it captures a real transition from a project-trained embedding model to a more production-oriented pretrained recognition stack.

Table 3: Differences Between the Offline Experimental Pipeline and the Deployed Web Engine

Parameter / Feature	Offline Pipeline (video_auth_pipeline.py)	Current Web Engine (buffalo_auth_core.py)
Identity embeddings	Custom CelebA CNN, 256-D	Buffalo / InsightFace, 512-D
Liveness threshold	0.1199	0.32
Minimum live ratio	0.60	0.40
Match threshold	0.60	0.55
Minimum match ratio	0.50	0.40
Maximum detected faces	48	6
Maximum sampled faces	12	3
Minimum detected faces	6	2
Purpose	Offline analysis / summary generation	Interactive webcam registration and authentication

4.4 Threshold and Runtime Differences

The offline pipeline and the deployed dashboard do not use identical thresholds or identical face-sampling policies. Table 3 summarizes the differences.

5 Hardware and Software Requirements, Installation, and Demo Execution on a Separate PC/Laptop

This section mirrors the deployment guidance written in the project README. The same requirements, installation routes, commands, and troubleshooting advice are intentionally duplicated here so that the report is self-contained.

5.1 Hardware Requirements

5.1.1 Minimum Hardware

- 64-bit Windows laptop or PC,
- Intel Core i5 / AMD Ryzen 5 class CPU or better,
- 8 GB RAM,
- 5 GB free disk space,

- integrated or external webcam,
- acceptable ambient lighting.

5.1.2 Recommended Hardware

- Windows 10 or Windows 11, 64-bit,
- Intel Core i7 / AMD Ryzen 7 or better,
- 16 GB RAM,
- 10 GB free disk space,
- 720p or 1080p webcam,
- optional NVIDIA GPU for faster inference.

The project can run on CPU, so a discrete GPU is not mandatory for classroom demonstration.

5.2 Software Requirements

5.2.1 Recommended Software Stack

- Windows 10/11 x64,
- Python 3.13 x64,
- `pip`,
- Google Chrome or Microsoft Edge,
- browser webcam permission enabled.

5.2.2 Why Python 3.13 Is Recommended

This repository contains a local `.deps/` directory with compiled Windows Python packages matching CPython 3.13. Therefore, the easiest way to run the copied repository on another PC/laptop without rebuilding those binaries is:

- Windows x64,
- Python 3.13 x64.

5.2.3 Minimum External Packages for the Web Demo

For the current web dashboard, the minimum external packages that should be installed are:

```
pip install flask torch
```

5.2.4 Optional Packages for Offline Scripts and Notebooks

If the evaluator also wants to run the offline pipelines and notebooks, the recommended additional packages are:

```
pip install torchvision tqdm pyyaml jupyter
```

5.2.5 Optional Full Fresh Install

If the evaluator chooses not to rely on the bundled `.deps/` folder, a fuller environment can be installed with:

```
pip install flask torch torchvision pillow numpy tqdm pyyaml  
opencv-python-headless insightface onnxruntime
```

5.2.6 Additional Windows Runtime Dependency

If DLL-related import problems appear for OpenCV or ONNX Runtime [8,9], installing the Microsoft Visual C++ Redistributable 2015–2022 (x64) is recommended.

5.3 Files That Must Be Copied

To run the demo on a separate PC/laptop, the entire project folder should be copied. At minimum, the following folders and files must remain intact:

- `buffalo/`,
- `checkpoints/`,
- `face_liveness_vit/`,
- `templates/`,
- `app_data/`,
- `.deps/`,
- `.vendor/`,

- `video_auth_webapp.py`,
- `buffalo_auth_core.py`,
- `run_dashboard.py`.

If any of these are missing, the demo may fail because of absent templates, model files, database files, or runtime dependencies.

5.4 Installation on a Separate PC/Laptop

5.4.1 Option A: Recommended Installation for This Repository

This is the preferred installation path for demonstration because it matches the current repository structure.

Step 1: Install Python

Install Python 3.13 x64 and verify:

```
python -version
pip -version
```

Step 2: Open the Project Root

Open PowerShell in the folder containing `run_dashboard.py`, `video_auth_webapp.py`, and `buffalo_auth_core.py`.

Step 3: Create a Virtual Environment (Recommended)

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Step 4: Upgrade Pip

```
python -m pip install -upgrade pip
```

Step 5: Install Minimum Packages

```
pip install flask torch
```

Step 6: Start the Dashboard

```
python run_dashboard.py
```

If successful, Flask should serve the application at `http://127.0.0.1:5000`.

Step 7: Open the Browser

Open:

```
http://127.0.0.1:5000
```

and allow camera access when prompted.

5.4.2 Option B: Fresh Environment Without Relying on Bundled Runtime Packages

This is the alternative path when the evaluator wants a more standard Python installation.

Step 1: Create and activate a virtual environment

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Step 2: Install a full dependency set

```
python -m pip install -upgrade pip
pip install flask torch torchvision pillow numpy tqdm pyyaml
opencv-python-headless insightface onnxruntime
```

Step 3: Run the dashboard

```
python run_dashboard.py
```

5.5 How to Run and Demonstrate the Project

The shortest presentation path on another laptop is:

1. open the Home page,
2. explain the Secure Verification Flow,
3. open Register,
4. enter a username,
5. start the webcam,
6. complete the live challenge,
7. allow the system to auto-capture frames,
8. show the enrollment result page,

9. open Authenticate,
10. complete a new live challenge,
11. show the final authentication decision,
12. open Audit,
13. show the event history,
14. optionally open Unknown Alerts and explain its purpose.

For quick startup, the minimal command sequence is:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -upgrade pip
pip install flask torch
python run_dashboard.py
```

5.6 Browser and Webcam Notes

- Use Chrome or Edge for the smoothest webcam experience.
- Allow webcam access in browser settings.
- The current web demo is designed for live webcam capture; direct video upload is disabled.
- Better lighting and a steady face position improve challenge verification and liveness performance.

5.7 Expected Files Updated During a Demo

Running the demo may update:

- app_data/auth.db,
- app_data/uploads/,
- app_data/profiles/.

Because the inspected repository already contains data, an evaluator who wants a completely fresh demonstration should back up the current `app_data/` directory first.

5.8 Troubleshooting

5.8.1 Missing Flask

If `ModuleNotFoundError`: No module named 'flask' appears:

```
pip install flask
```

5.8.2 Missing PyTorch

If `ModuleNotFoundError`: No module named 'torch' appears:

```
pip install torch
```

5.8.3 OpenCV / ONNX / InsightFace Import Errors

Common causes are:

- using the wrong Python version,
- missing Visual C++ runtime,
- incomplete project copy,
- missing `.deps/` folder.

Recommended fixes:

1. confirm Python 3.13 x64,
2. confirm the `.deps/` folder exists,
3. install Microsoft Visual C++ Redistributable x64,
4. if needed, switch to the full-install Option B path.

5.8.4 Webcam Does Not Start

Check:

- browser camera permission,
- whether another application is using the webcam,
- Windows privacy settings for camera access.

5.8.5 Registration or Authentication Fails Repeatedly

Common reasons include:

- weak lighting,
- face not centered,
- challenge not completed clearly,
- blurry frames,
- liveness or identity thresholds not met.

5.8.6 Port 5000 Already in Use

Stop the conflicting process and rerun:

```
python run_dashboard.py
```

6 Security Controls and Operational Behavior

6.1 HTTP and Session Security

The web app implements several practical security measures using Flask [5]:

- CSRF token generation and verification,
- challenge nonce validation,
- challenge expiration,
- rate limiting by flow,
- temporary lockout after repeated failures,
- security headers such as `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`, and `Cache-Control: no-store`.

The configured limits in the current code are:

- registration: 5 attempts per 10 minutes,
- authentication: 10 attempts per 10 minutes,
- lockout after 5 failed attempts for 10 minutes.

6.2 Challenge Verification

Challenge verification is intentionally duplicated:

- the browser estimates blink / turn completion using MediaPipe landmarks,
- the backend re-verifies the challenge from captured frames using OpenCV and face landmarks.

This reduces the chance that a client-side-only claim is trusted blindly.

6.3 Privacy and Audit Behavior

The dashboard does not publicly expose enrolled identities. Failed authentication results hide the resolved identity name even when a best internal candidate exists. Audit records are stored in SQLite [10], and a separate unknown-attempt queue is available for live sessions that fail identity matching without looking spoofed.

7 Strengths, Limitations, and Recommendations

7.1 Strengths

The inspected project has several strong qualities:

- layered authentication logic rather than plain face recognition,
- practical use of challenge-response verification,
- privacy-first user interaction,
- clear separation between spoof-like sessions and unknown real visitors,
- bundled model artifacts that support offline local deployment,
- preserved notebooks showing the research evolution of the system.

7.2 Limitations

The repository also reveals some limitations:

- two parallel identity pipelines exist, which can cause maintenance drift,
- the manifest for evaluation videos is present, but the actual raw MP4 files are absent in this snapshot,
- threshold sweep scripts exist, but their generated report files are not committed,

- the frontend defines image-quality analysis helpers, but the capture loop is not as strongly gated by them as the interface text suggests,
- profile embeddings are stored locally without evidence of extra template encryption at rest.

7.3 Recommendations

The most useful next improvements would be:

- unify the offline and web identity pipelines,
- commit a formal dependency manifest,
- publish reproducible evaluation outputs,
- add standard biometric metrics such as FAR, FRR, EER, and ROC,
- harden biometric template storage for stronger at-rest security.

8 Conclusion

SecureFace is a well-developed local biometric security demo rather than a simple face-recognition script. Its current deployed path combines Flask, browser webcam capture, challenge-response verification, a Vision Transformer liveness model, Buffalo face embeddings, SQLite audit logging, and a privacy-aware decision interface. The repository also preserves its earlier research path, including a custom CelebA CNN embedding model and evaluation utilities. This makes the project valuable both as a working classroom demo and as a study of iterative system design.

Equally important, the repository can be demonstrated on a separate PC/laptop with practical effort. The hardware requirements are moderate, the software requirements are explicit, and the demo can be run locally by starting the dashboard and allowing browser webcam access. By including the same setup content in both the README and this report, the project documentation becomes directly usable for academic submission, lab evaluation, and live presentation.

References

- [1] A. Dosovitskiy *et al.*, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *Proc. ICLR*, 2021. [Online]. Available: <https://openreview.net/forum?id=YicbFdNTTy>
- [2] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proc. IEEE/CVF CVPR*, 2016, pp. 770–778. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html
- [3] DeepInsight, “InsightFace: 2D and 3D Face Analysis Project,” GitHub repository. [Online]. Available: <https://github.com/deepinsight/insightface>
- [4] Y. Zhang *et al.*, “CelebA-Spoof: Large-Scale Face Anti-Spoofing Dataset with Rich Annotations,” in *Proc. ECCV*, 2020, pp. 70–85. [Online]. Available: https://www.ecva.net/papers/eccv_2020/papers_ECCV/papers/123570069.pdf
- [5] Pallets, “Flask Documentation.” [Online]. Available: <https://flask.palletsprojects.com/>
- [6] Google, “MediaPipe Face Mesh Documentation.” [Online]. Available: https://mediapipe.readthedocs.io/en/latest/solutions/face_mesh.html
- [7] A. Paszke *et al.*, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019. [Online]. Available: <https://arxiv.org/abs/1912.01703>
- [8] Microsoft, “ONNX Runtime Documentation.” [Online]. Available: <https://onnxruntime.ai/docs/>
- [9] G. Bradski, “The OpenCV Library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [10] SQLite Consortium, “SQLite Documentation.” [Online]. Available: <https://sqlite.org/docs.html>
- [11] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *Proc. IEEE/CVF CVPR*, 2019, pp. 4690–4699. [Online]. Available: https://openaccess.thecvf.com/content_CVPR_2019/html/Deng_ArcFace_Additive_Angular_Margin_Loss_for_Deep_Face_Recognition_CVPR_2019_paper.html